hetero1d

hetero1d explained

Classes, arguments, materials and the absolute energy scale: how it works and why

Ravi Kumar

Explainer and reference, package version 0.1.4

November 2026

github.com/erw-del/hetero1d · pypi.org/project/hetero1d

Contents

1. About this document
2. The design: three separate ideas
3. Material functions
4. The Layer class
5. The Structure class
6. The Solver class: setting up and solving
7. The Solver class: analysis, fields and output
8. The potentials module
9. Band edges and the absolute energy scale
10. Limits of the model
Appendix: quick reference

For every part of hetero1d this document explains what it is, how to use it with all its arguments, and why it was designed that way.

1. About this document

hetero1d is built from a small number of parts: three classes, four material functions, a module of ready-made potentials and a materials database. This document explains each part in detail: **what** it is, **how** to use it with every argument and its default value, and **why** it was designed that way. The last chapters explain how band edges, band gaps and offsets are calculated on hetero1d's absolute energy scale, with worked examples you can check by hand.

It complements the user guide (which teaches the physics and the workflows) and the beginner tutorial notebook (which teaches Python). Here the focus is the design and the reasoning behind each choice.

Name	Kind	Purpose	Chapter
material()	function	parameters of a material at a temperature	3
list_materials()	function	print every available material	3
add_material()	function	register your own material	3
lattice_matched()	function	composition with zero lattice mismatch	3
Layer	class	one layer of a structure	4
Structure	class	the layer stack: your sample	5
Solver	class	the calculation and all its results	6, 7
potentials	module	ready-made potentials for V_ext	8
BINARIES, TERNARIES, QUATERNARIES, SHAPES	dictionaries	the material database and the graded-well shapes	3, 5

Everything is imported in one line:

```
from hetero1d import Structure, Layer, Solver, material, add_material, list_materials,
lattice_matched, potentials
```

2. The design: three separate ideas

hetero1d separates three ideas that physicists already keep separate in the lab: the **material data** (a handbook), the **sample** (a growth recipe) and the **measurement** (an experiment that produces results).

Part	Holds	Lab analogy
material()	band gap, offsets, masses, dielectric constants … of one material	a page of a data handbook
Structure	which layers, how thick, which composition, which doping, in which order	the growth recipe of the sample
Solver	the band profile on a grid, the conditions (temperature, fields), the computed states and everything derived from them	the experiment and its lab book

Why the Structure and the Solver are separate

- **One sample, many conditions.** The same Structure can be solved at 4 K and at 300 K, with and without an electric field, with a coarse and a fine grid. Each Solver is one set of conditions; the Structure stays untouched.
- **Clear responsibilities.** The Structure only describes geometry and materials; it knows nothing about grids or quantum mechanics. The Solver only calculates; it never changes the sample. This makes both easier to understand and to test.
- **Results stay with their conditions.** Because energies and wavefunctions are stored inside the Solver that produced them, you can never confuse the results at 4 K with those at 300 K.

```
s = Structure.single_well(well="GaAs", barrier=("AlGaAs", 0.3), Lw=10, Lb=25)   # one
sample
cold = Solver(s, T=4.2).solve()                                                  #
experiment 1
warm = Solver(s, T=300).solve()                                                  #
experiment 2
biased = Solver(s, T=300, field=50).solve()                                      #
experiment 3
```

Why named arguments

Almost every input is given by name, such as `Lw=10` or `T=4.2`, instead of by position. A line like `Solver(s, T=4.2, dz=0.05)` documents itself: a reader sees what every number means without consulting the manual. Named arguments can be written in any order, and every argument you leave out takes a sensible default value (listed in the tables of the following chapters), so short calls work out of the box and long calls stay readable.

Why units are fixed

Every quantity has one fixed unit: lengths in nm, energies in eV, temperature in K, electric field in kV/cm, magnetic field in tesla, volume doping in cm^{-3} and sheet density in cm^{-2}. These are the units semiconductor physicists use daily, so numbers can be typed exactly as they appear in papers, and there are no unit conversions to forget.

3. Material functions

3.1 material(name, x=None, y=None, T=300.0)

What it does: returns the parameters of one material at temperature T as a dictionary (a set of labelled values).

Argument	Meaning	Default	Example
name	material name: binary, ternary, quaternary or custom	required	"GaAs", "AlGaAs", "InGaAsP"
x	composition of a ternary, or first composition of a quaternary	None	x=0.3
y	second composition (quaternaries only)	None	y=0.85
T	temperature in K	300.0	T=4.2

```
material("GaAs")                        # binary
material("AlGaAs", x=0.3)               # ternary: Al0.3Ga0.7As
material("InGaAsP", x=0.4, y=0.85)      # quaternary: Ga0.4In0.6As0.85P0.15
material("GaAs", T=4.2)["Eg"]           # one value: the band gap at 4.2 K
```

The returned parameters

Key	Meaning	Unit	GaAs (300 K)
name	label		GaAs
Eg	band gap (Γ valley) at T	eV	1.4225
Ev	valence-band edge on the absolute scale	eV	-0.800
Ec	conduction-band edge = Ev + Eg	eV	0.6225
me	electron effective mass	m_0	0.067
mhh	heavy-hole mass along z (growth axis)	m_0	0.350
mlh	light-hole mass along z	m_0	0.090
mhh_par	heavy-hole mass in the layer plane	m_0	0.111
mlh_par	light-hole mass in the layer plane	m_0	0.204
eps_s	static dielectric constant		12.90
eps_inf	high-frequency dielectric constant		10.89
hwLO	LO-phonon energy	eV	0.03625
a	lattice constant	Å	5.65325
Ep	Kane energy	eV	28.8
Dso	spin-orbit splitting	eV	0.341
g	electron g-factor (Roth formula)		-0.61
approx	True for approximate parameters		False

Why each parameter is needed

Parameter	Used for
Ec, Ev (and Eg)	the potential in which electrons and holes are confined; band offsets; transition energies

Parameter	Used for
me, mhh, mlh	confinement energies along z: a lighter mass gives higher levels
mhh_par, mlh_par	motion parallel to the layers: in-plane magnetic field, exciton reduced mass, density of states
eps_s	screening of the Coulomb interaction (excitons) and Poisson's equation (doping)
eps_s, eps_inf, hwLO	strength and threshold of LO-phonon scattering (factor 1/ε∞ - 1/εs)
a	lattice mismatch check (describe) and lattice matching
Ep, Dso	the electron g-factor (Zeeman splitting)

Why there are two hole masses for each kind of hole

The valence band of III-V semiconductors is not isotropic. The hole masses follow from the Luttinger parameters γ_1 and γ_2, and they differ along the growth direction and in the plane:

$$m_{hh,\,z} = \frac{1}{\gamma_1 - 2\gamma_2}, \quad m_{lh,\,z} = \frac{1}{\gamma_1 + 2\gamma_2}, \quad m_{hh,\,\parallel} = \frac{1}{\gamma_1 + \gamma_2}, \quad m_{lh,\,\parallel} = \frac{1}{\gamma_1 - \gamma_2}$$

Along z, the heavy hole really is heavy (0.35 m_0 in GaAs). In the plane, the order reverses: the in-plane heavy-hole mass (0.11 m_0) is *lighter* than the in-plane light-hole mass (0.20 m_0). This "mass reversal" is real physics of the valence band. hetero1d uses the z masses for confinement and the in-plane masses where motion in the plane matters.

Why the g-factor is calculated, not tabulated

$$g = 2 - \frac{2\,E_p\,\Delta_{so}}{3\,E_g\,(E_g + \Delta_{so})}$$

The two-band Roth formula gives g from Eg, Ep and Δso. Because Eg depends on temperature and composition, g does too: g(GaAs) is -0.32 at 4 K and -0.61 at 300 K, and in AlGaAs it changes sign at a small aluminium fraction (x ≈ 0.07 at 4 K and 0.1 at 300 K in this model; measured: about 0.12). A fixed table could not capture this; the formula does it automatically for every alloy.

3.2 list_materials()

What it does: prints every material in the database and the meaning of the composition parameters. It takes no arguments.

```
list_materials()
```

Group	Materials	Composition
Binaries (12)	GaAs, AlAs, InAs, InP, GaSb, AlSb, GaP, AlP, InSb, GaN, AlN, InN	none
Ternaries (17)	AlGaAs, InGaAs, InAlAs, GaAsSb, AlGaSb, InGaP, InAsSb, InGaSb, InAlSb, AlAsSb, InAsP, GaAsP, AlGaP, InAlP, AlGaN, InGaN, InAlN	x = fraction of the first element named (Al in AlGaAs, In in InGaAs, Sb in GaAsSb, As in InAsP)
Quaternaries (4)	InGaAsP, InGaAsSb	Ga_x In_(1-x) As_y P_(1-y) (or Sb_(1-y)): x = Ga, y = As
	AlGaInAs, AlGaInP	Al_x Ga_y In_(1-x-y) As (or P): x = Al, y = Ga

Why it exists: the meaning of x is a common source of confusion (is x the Al or the Ga fraction?). Printing the convention for every alloy removes the guesswork. The nitrides are flagged as approximate: they use zinc-blende parameters without the polarisation fields of real wurtzite material.

3.3 add_material(...)

What it does: registers a material with your own parameters, which can then be used by name in any structure.

```
add_material(name, Ec, Ev, me, mhh, mlh, eps_s, eps_inf=None, hwLO=0.036, a=5.65325,
g=2.0, mhh_par=None, mlh_par=None)
```

Argument	Meaning	Default
name	the name you will use in structures	required
Ec, Ev	band edges on hetero1d's absolute scale (eV); Eg = Ec - Ev	required
me, mhh, mlh	electron, heavy-hole and light-hole masses along z (m_0)	required
eps_s	static dielectric constant	required
eps_inf	high-frequency dielectric constant	same as eps_s
hwLO	LO-phonon energy (eV)	0.036 (GaAs-like)
a	lattice constant (Å)	5.65325 (GaAs)
g	electron g-factor	2.0
mhh_par, mlh_par	in-plane hole masses	same as mhh, mlh

```
add_material("MyBarrier", Ec=0.95, Ev=-0.98, me=0.092, mhh=0.38, mlh=0.11, eps_s=12.2)
s = Structure().add("MyBarrier", 20).add("GaAs", 8).add("MyBarrier", 20)
```

Why it exists

- **Reproducing a paper exactly:** papers often use their own parameter sets. Registering them as custom materials makes a comparison like-for-like. The reproduction of Harrison & Valavanis used exactly this.
- **Calibrating to your own samples:** if photoluminescence shows your barriers are slightly different from the database, enter measured values.
- **Materials not in the database:** II-VI compounds, other alloys, or hypothetical materials for what-if studies.

Why Ec and Ev are given instead of Eg

A material only makes sense in a heterostructure together with its band alignment to its neighbours. Giving Ec and Ev on the same absolute scale as the built-in materials (Chapter 9) fixes both the gap and the offsets in one step. For example, to make a barrier 300 meV above GaAs in the conduction band, set Ec = 0.6225 + 0.300 = 0.9225 eV.

> **Caution.** Custom materials do not change with temperature: their values are used exactly as given at every T. If you need temperature dependence, register one material per temperature, or calculate the shifted values yourself.

3.4 lattice_matched(name, substrate="InP", fixed=None, T=300.0)

What it does: finds the alloy composition whose lattice constant equals that of the substrate.

Argument	Meaning	Default
name	ternary or quaternary alloy	required
substrate	material to match	"InP"
fixed	for quaternaries: which composition is fixed, as ("x", value) or ("y", value)	None
T	temperature	300.0

```
lattice_matched("InGaAs", "InP")                        # 0.534 -> In0.534Ga0.466As
lattice_matched("InGaAsP", "InP", fixed=("y", 0.9))    # quaternary: fix As = 0.9,
find Ga
lattice_matched("AlGaInP", "GaAs", fixed=("x", 0.25))  # fix Al = 0.25, find Ga
```

How it works: the lattice constant of an alloy is interpolated linearly (Vegard's law), and a root-finding algorithm solves a(x) = a_substrate.

Why it matters: a layer grows without defects only if its lattice matches the substrate. Mismatched layers are strained, and beyond a critical thickness they relax by forming dislocations that destroy device performance. Because strain is not modelled in hetero1d, lattice-matched designs are also the ones it calculates most accurately. A quaternary has two composition variables, so one must be fixed; the other is then determined by the matching condition, leaving one free parameter to tune the band gap.

4. The Layer class

```
Layer(material, thickness, x=None, doping=0.0, y=None, label=None)
```

Argument / attribute	Meaning	Default
material	material name	required
thickness	thickness in nm	required
x, y	compositions	None
doping	donor density in cm^{-3} (negative = acceptors)	0.0
label	name that groups thin slices of a graded section into one region	None

How you use it: usually indirectly. `Structure.add()` and the builders create Layer objects for you. You read them through `s.layers`:

```
for layer in s.layers:
    print(layer.material, layer.thickness, layer.x, layer.doping)
print(s.layers[1].thickness)          # thickness of layer number 1 (counting from 0)
```

Why a separate class: each layer is a small, self-contained record. Keeping it separate means a structure is simply an ordered list of such records, which is easy to inspect, modify and print. The `label` exists for graded structures: a parabolic well is built from 200 slices of 0.2 nm, and the label lets hetero1d treat them as one region in `describe()` and in `region_probabilities()`.

Doping is assumed fully ionised. Donors give electrons to the structure; the Schrödinger-Poisson solver places those electrons self-consistently (Chapter 7).

5. The Structure class

```
Structure(layers=None, substrate="GaAs", name="structure")
```

Argument	Meaning	Default
layers	optional starting list of Layer objects	None (empty)
substrate	material for the lattice-mismatch check	"GaAs"
name	label shown by describe()	"structure"

Attributes: `layers` (the list of Layer objects, left to right), `substrate`, `name`, `period` (one period, set by superlattice builders and used by minibands), `total_thickness` (nm) and `regions` (graded sections grouped).

Why left to right with z = 0 at the left edge: it matches how growth sheets are written (substrate side first) and gives every position an unambiguous coordinate, used by plots, by V_ext and by the field.

5.1 Building layer by layer

Method	Arguments (defaults)	What it does
add	material, thickness, x=None, doping=0.0, y=None	append one layer; returns the structure, so calls can be chained
add_graded	material, thickness, x=None, y=None, slice_nm=0.2, doping=0.0, label=None	append a layer whose x and/or y vary with position t (nm); x and y can be numbers or functions
add_shaped_well	shape, Lw, alloy="AlGaAs", x_center=0.0, x_edge=0.3, slice_nm=0.2, label=None	append one graded well of a named or custom shape
describe	T=300.0	print the layer table with band edges, masses, g-factor and mismatch

```
s = Structure(name="my sample", substrate="GaAs")
s.add("AlGaAs", 30, x=0.3).add("GaAs", 10).add("AlGaAs", 30, x=0.3)           #
chained
s.add_graded("AlGaAs", 10, x=lambda t: 0.3 * t / 10)                          #
linear grading 0 -> 0.3
```

Why chaining: `add` returns the structure itself, so short stacks fit on one readable line. **Why graded layers are made of slices:** the solver needs a material at every grid point. Slicing a smooth profile into 0.2 nm steps, finer than the default 0.1 nm grid spacing, represents it accurately while reusing the same material machinery as ordinary layers.

5.2 Builders

Builders create common structures in one line. They are called on the word `Structure` itself (they are class methods), and all accept `substrate=` and `name=` in addition to the arguments below.

Builder	Arguments and defaults
single_well	well=("GaAs",), barrier=("AlGaAs", 0.3), Lw=10.0, Lb=20.0
multi_well	n_wells, well, barrier, Lw=10.0, Lb=5.0, Lb_out=20.0 (Lw, Lb may be lists)
double_well	well, barrier, Lw=10.0 (or (w1, w2)), Lb=2.0, Lb_out=20.0
superlattice	n_periods, well=("GaAs",), barrier=("AlAs",), Lw=5.0, Lb=2.0, Lb_out=20.0

Builder	Arguments and defaults
step_well	well, step=("AlGaAs", 0.15), barrier=("AlGaAs", 0.3), Lw=6.0, Ls=6.0, Lb=20.0
parabolic_well	Lw=40.0, Lb=20.0, alloy="AlGaAs", x_center=0.0, x_edge=0.3
triangular_well	Lw=20.0, Lb=20.0, alloy, x_center, x_edge, wall="left" (or "right")
v_well	Lw=30.0, Lb=20.0, alloy, x_center, x_edge
graded_well	shape="parabolic", Lw=40.0, Lb=20.0, alloy, x_center, x_edge, slice_nm=0.2
multi_graded_well	n_wells, shape="parabolic", Lw=20.0, gap=5.0, Lb_out=20.0, alloy, x_center, x_edge
sawtooth	n_wells=4, Lw=20.0, gap=0.0, Lb_out=20.0, alloy, x_center, x_edge
shaped_well	shape, n_wells=1, Lw=40.0, alloy, x_min=0.0, x_max=0.3, Lb=20.0, gap=0.0, slice_nm=0.2 (general builder)
from_profile	alloy, z, x, Lb=0.0, x_barrier=None (arrays: measured profile)

Why the defaults are what they are

- **GaAs/$Al_{0.3}Ga_{0.7}As$:** the most studied and best-characterised system, lattice-matched, with a well-known 259 meV conduction-band offset; a safe starting point for learning and validation.
- **Lb = 20-25 nm outer barriers:** thick enough for the bound states of typical wells to decay before the edge of the calculation box (Section 6.1); weakly bound states need more.
- **x_edge = 0.3:** keeps AlGaAs a direct semiconductor (it becomes indirect above x ≈ 0.45, outside the Γ-valley model).

Why graded shapes use x_center and x_edge

Every shaped well maps a relative position u = 0 … 1 across the well to a shape value from 0 (well bottom) to 1 (barrier), and then to a composition between x_center and x_edge. This one rule covers both kinds of alloys: in AlGaAs the well has less aluminium (x_center = 0, x_edge = 0.3), in InGaAs the well has more indium (x_center = 0.2, x_edge = 0). The named shapes in SHAPES are square, parabolic, triangular, triangular_right, v, half_parabolic and cosine; any function of u works as well.

6. The Solver class: setting up and solving

```
Solver(structure, T=300.0, field=0.0, dz=0.1, Bz=0.0, Bpar=0.0, z0=None, V_ext=None)
```

Argument	Meaning	Unit	Default
structure	the Structure to calculate		required
T	temperature: sets the band gaps (Varshni law)	K	300.0
field	electric field along z; positive raises the bands to the right	kV/cm	0.0
dz	grid step: smaller = more grid points	nm	0.1
Bz	magnetic field along the growth axis	T	0.0
Bpar	magnetic field in the layer plane	T	0.0
z0	orbit centre for the in-plane field	nm	centre of the structure
V_ext	extra potential: a function V(z) returning eV		None

6.1 What happens when a Solver is created

1. **A grid is laid over the structure.** The number of points is set by the grid step:
2. **Every grid point takes the parameters of its layer:** Ec, Ev, the masses, the dielectric constant, the g-factor and the doping.
3. **External potentials are added** to both band edges (an electrostatic potential moves the conduction and valence bands together):

$$N_{points} = \frac{L_{total}}{dz} + 1$$

$$E_c(z) = E_c^{mat}(z) + eF(z - z_c) + V_{ext}(z) + V_H(z), \qquad E_v(z) = E_v^{mat}(z) + eF(z - z_c) + V_{ext}(z) + V_H(z)$$

The terms are the electric field (zero at the centre z_c of the structure), the user potential V_ext, and the Hartree potential V_H from a Schrödinger-Poisson calculation. An in-plane magnetic field adds a diamagnetic term when the states are solved.

Available right after creation: `z`, `dz`, `Ec`, `Ev` (with fields and potentials), `Ec0`, `Ev0` (without them), `me`, `mhh`, `mlh`, `mhh_par`, `mlh_par`, `g`, `eps`, `Nd`, `layer_index` and `params` (the material dictionary of each layer).

Why these defaults

- **T = 300 K:** most devices operate at room temperature. Excitons, magneto-transport and many optical experiments use 4.2 K or 77 K; set T explicitly for them.
- **dz = 0.1 nm:** for a 10 nm GaAs well the error against the exact solution is about 0.01 meV, far below experimental uncertainty, and the calculation takes milliseconds. Thin wells, thin barriers and excitons benefit from 0.02-0.05 nm.
- **Fields default to zero:** the structure is solved as grown unless you ask otherwise.

Why the wavefunction is zero at the edges

The calculation box ends where the structure ends, and the wavefunction is set to zero there (as if infinitely high walls enclosed the structure). Bound states have decayed long before the edges if the outer barriers are thick enough. States above the barrier top become standing waves in the box: they

represent the continuum, and their energies depend on the total thickness. That is why outer barriers of 20-40 nm are recommended.

6.2 solve(n_e=6, n_h=6)

Argument	Meaning	Default
n_e	number of electron states to compute	6
n_h	number of heavy-hole states and of light-hole states	6

What it does: solves the BenDaniel-Duke equation three times: electrons in Ec(z) with the electron mass, heavy holes in -Ev(z) with the heavy-hole mass, light holes in -Ev(z) with the light-hole mass. The finite-difference matrix is tridiagonal and is solved exactly with LAPACK, which is fast and gives identical results in every run.

What it stores: `Ee`, `Ehh`, `Elh` (energies, eV, absolute scale), `psi_e`, `psi_hh`, `psi_lh` (normalised wavefunctions: one column per state on the grid `z`, in $nm^{-1/2}$) and `e_bound`, `hh_bound`, `lh_bound` (True for bound states). It returns the solver itself, which is why `Solver(s).solve()` works on one line.

Why holes use -Ev: turning the valence band upside down lets holes be treated exactly like electrons, with positive energies increasing downward into the valence band. The results are converted back to the absolute scale afterwards, so hole energies lie below Ev.

Why compute extra states: they cost almost nothing, they show unbound states for comparison, and under strong electric fields some of the lowest eigenstates can be spurious states in the tilted outer barriers. Field sweeps therefore compute more states and pick the ones inside the wells with `active_states()`.

6.3 schrodinger_poisson(n_e=8, mix=0.1, tol=1e-5, max_iter=600, verbose=True, right_bc="neumann")

Argument	Meaning	Default
n_e	starting number of electron states (increased automatically)	8
mix	fraction of the new potential mixed in each iteration	0.1
tol	convergence threshold for the potential (eV)	1e-5
max_iter	maximum number of iterations	600
verbose	print a summary when finished	True
right_bc	"neumann" (zero field) or "dirichlet" (V = 0) at the right end	"neumann"

What it does: in doped structures the electrons and ionised donors create a field that bends the bands. The solver alternates between the Schrödinger equation and Poisson's equation until the potential stops changing, then stores `VH` (Hartree potential), `n3d` (electron density, nm^{-3}) and `EF` (Fermi level).

Why mixing: using the new potential directly makes the iteration oscillate (too many electrons push the band up, which empties the well, which pulls it down). Mixing in only 10 % each time damps this. **Why automatic states:** with too few states, electrons that belong to higher states would be forced into the lowest ones; the solver adds states until the highest lies 10 k_BT above E_F. **Why zero field at the right end by default:** a real sample sits on a thick undoped buffer where the field dies away; pinning the potential to zero there would create an artificial pocket for electrons.

7. The Solver class: analysis, fields and output

7.1 Analysis methods

Method	Arguments (defaults)	Returns	Why it is useful
layer_probabilities	psi	probability in each layer	which well a state belongs to; leakage into barriers
region_probabilities	psi	list of (region, probability)	the same for graded structures, by region
expect_z	psi	⟨z⟩ and spread (nm)	position of a state; electron-hole separation in a field
mass_average	psi, m	averaged mass	the effective mass a state really feels (well and barrier)
active_states	psi, threshold=0.5	indices of well states	skip spurious states under strong fields
intersubband	bound_only=True	dicts: i, j, dE, lam_um, z, f, m	infrared detectors, cascade lasers
isb_absorption	energies, n2d_cm2=1e11, gamma=0.005	absorption per pass (fraction)	detector design, doping choice
interband	bound_only=True	dicts: e, h, band, E, lam_nm, overlap2, weight	laser and LED wavelengths, selection rules
interband_absorption	energies, gamma=0.005	relative absorption	absorption edge, modulators
exciton	ie=0, ih=0, hole="hh"	dict: Eb_meV, lambda_nm, E_transition_eV, mu, eps	PL peak position, exciton binding
fermi_level	n2d_cm2, states=None	E_F (eV), populations (cm^{-2})	doped wells, absorption, transport
lo_phonon_rates	(none)	dicts: i, f, dE, rate, tau_ps	excited-state lifetimes, carrier dynamics
transmission	energies, V=None, m=None	T(E)	tunnelling, resonant-tunnelling diodes
minibands	energies, period=None, dz=0.02	q, d, list of (E_min, E_max)	superlattice band structure

Why results come as dictionaries: a transition has several properties (energy, wavelength, strength). A dictionary keeps them together with readable labels, such as `t["lam_nm"]`, so the meaning of every number is visible in the code.

Why `bound_only=True`: transitions to unbound states depend on the size of the calculation box and would mislead; they are excluded unless you ask for them.

7.2 Field methods

Method	Arguments (defaults)	Returns
stark_sweep	fields, n_e=4, n_h=4	dict of arrays versus field (energies, overlaps, dipoles, positions)
g_factor	n	g-factor of subband n
zeeman	B=None	spin splitting of each subband (eV)
landau_levels	B=None, N_max=10, bound_only=True	dicts: n, N, s, E, hwc

Method	Arguments (defaults)	Returns
landau_dos	energies, B=None, gamma=0.001, N_max=60	Landau-level density of states
fermi_level_B	n2d_cm2, B=None, gamma=0.001, N_max=60	Fermi level in field B
landau_fan	B_values, n2d_cm2=None, N_max=8, gamma=0.001	levels, E_F, filling factor versus B
inplane_dispersion	ky_values, n_e=4	E_n(k_y) in an in-plane field

Why sweep methods exist: field-dependent physics (Stark effect, Landau fans) is always studied as a function of the field. The sweeps handle the repetition and, importantly, track the right states as the field changes, which is easy to get wrong by hand.

7.3 Output methods

Method	Arguments (defaults)	Output
report	n2d_cm2=None	all result tables printed
plot_summary	filename, n2d_cm2=1e11, gamma=0.005	six-panel overview figure (PNG)
plot_bands	ax, n_show=4, scale=0.05	band diagram into your own matplotlib axes
plot_stark	filename, fields, n_e=4, n_h=4	Stark-effect figure
plot_magnetic	filename, B_values, n2d_cm2=5e11, gamma=0.001, ky_max=0.2	Landau fan, DOS at E_F, in-plane dispersion

8. The potentials module

`V_ext` accepts any function of position. The `potentials` module provides the common textbook shapes so that you do not need to write the formulas yourself. Each function returns a potential V(z) in eV, with z in nm from the left edge.

Function	Arguments and defaults	Shape
parabolic	hw_meV=10.0, center=0.0, mass=0.067	harmonic oscillator with level spacing hw
triangular	field_kVcm=50.0, start=0.0, height=None	linear rise from a wall
v_shape	slope_kVcm=50.0, center=0.0, height=None	symmetric V
sawtooth	n=4, period=20.0, height=0.15, start=20.0, gap=0.0	n triangles
step	position=0.0, height=0.1	step
square_well	left=0.0, right=10.0, depth=0.2	square well
poschl_teller	depth=0.2, width=5.0, center=0.0	Pöschl-Teller well
gaussian	depth=0.1, width=5.0, center=0.0	Gaussian well
uniform_field	field_kVcm=10.0, z0=0.0	uniform field
combine	*potentials	sum of several potentials

```
box = Structure().add("GaAs", 200)
sol = Solver(box, V_ext=potentials.parabolic(hw_meV=10, center=100)).solve()     #
levels 5, 15, 25 meV
both = potentials.combine(potentials.parabolic(10, 100), potentials.uniform_field(20,
100))
```

Why two ways to make shaped wells

A grown parabolic well and a parabolic potential are physically different:

	Graded composition (Structure builders)	Formula potential (V_ext)
Physical meaning	the alloy changes with position	an electrostatic potential
Band gap	changes with position	unchanged
Conduction and valence band	move in opposite directions: electrons and holes both confined	move together: a well for electrons is a hill for holes
Effective mass	changes with composition	constant
Best for	real devices, optics, excitons	textbook models, gates, built-in fields, checking theory

Keeping both lets you test the solver against exactly solvable potentials (the harmonic oscillator, the triangular well and the Pöschl-Teller well are reproduced to better than 0.01 meV) and design real structures with the same tool.

9. Band edges and the absolute energy scale

9.1 Eg, Ec and Ev

In a semiconductor, electrons fill the valence band up to its top edge **Ev**. Above a gap with no states lies the conduction band, starting at its bottom edge **Ec**. The **band gap** is their difference:

$$E_g = E_c - E_v$$

Eg is the smallest energy that lifts an electron from the valence band into the conduction band; it sets the colour of light a bulk material absorbs and emits. Electrons in a quantum well live above Ec, holes below Ev.

9.2 Why one absolute scale

Knowing each material's band gap is not enough to describe a heterostructure. When two materials are grown together, what matters is **how their bands line up**: the band offsets ΔEc and ΔEv, which form the walls that confine electrons and holes.

$$\Delta E_c = E_c^B - E_c^A, \qquad \Delta E_v = E_v^A - E_v^B, \qquad \Delta E_c + \Delta E_v = E_g^B - E_g^A$$

If every material had its own private energy zero, the offsets would have to be specified separately for every pair of materials: with 33 materials that is hundreds of pairs. Instead, hetero1d places the valence-band edge of every material on **one common energy scale**, the valence-band-offset (VBO) scale of Vurgaftman, Meyer and Ram-Mohan (J. Appl. Phys. 89, 5815, 2001), on which **the InSb valence band is at 0 eV**. Each material then needs only one number, its VBO, and the offsets between *any* two materials follow by subtraction.

$$E_c = \mathrm{VBO} + E_g(T), \qquad E_v = \mathrm{VBO}$$

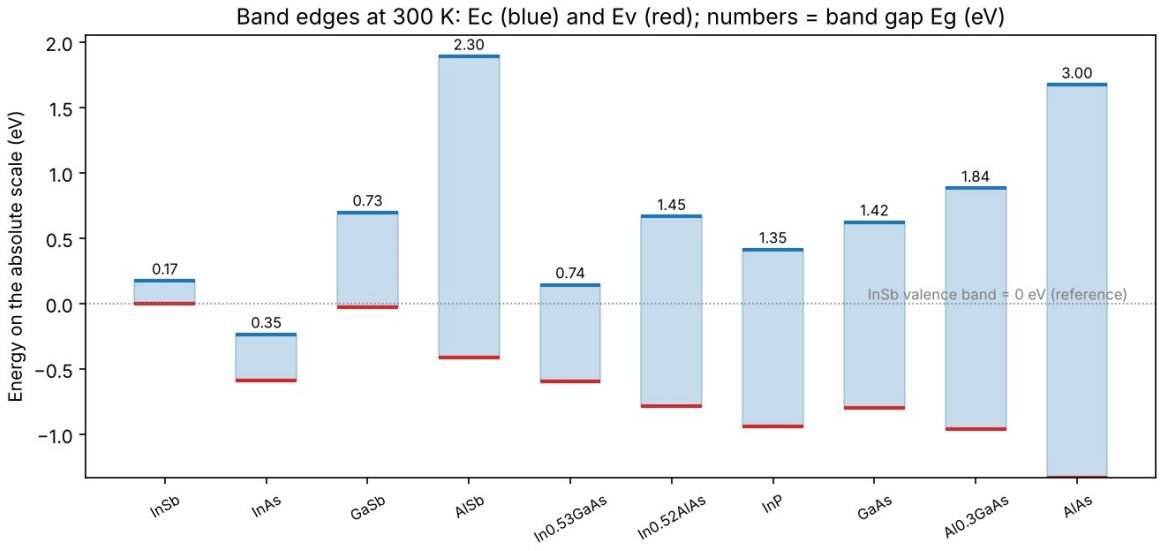


Figure 9.1: band edges of common III-V materials and alloys at 300 K on hetero1d's absolute scale. Blue lines are Ec, red lines Ev, the numbers are the band gaps. Any two materials can be compared directly: for example, InAs (Ec) lies below GaSb (Ev), the broken-gap alignment used in infrared superlattices.

9.3 Step 1: band gap versus temperature

Band gaps shrink as the lattice expands and vibrates. hetero1d uses the Varshni law with tabulated α and β for every binary:

$$E_g(T) = E_g(0) - \frac{\alpha T^2}{T + \beta}$$

Worked example (GaAs at 300 K): Eg(0) = 1.519 eV, α = 5.405×10^{-4} eV/K, β = 204 K. The correction is 5.405×10^{-4} × 300^2 / (300 + 204) = 0.0965 eV, so Eg = 1.4225 eV. With VBO = -0.800 eV: **Ev = -0.800 eV** and **Ec = -0.800 + 1.4225 = 0.6225 eV**.

Why the whole change goes into Ec: the VBO is taken as temperature independent, a standard and well-supported approximation. The valence band then stays fixed, and the conduction band moves down as the gap shrinks.

9.4 Step 2: alloys

An alloy such as Al_x Ga_(1-x) As is a random mixture of two binaries. Most parameters lie on a straight line between them, but the band gap (and for some alloys the VBO and the electron mass) bend away from it. The bending is described by a bowing parameter C:

$$P(x) = x\,P_A + (1 - x)\,P_B - x(1 - x)\,C$$

Worked example ($Al_{0.3}Ga_{0.7}As$ at 300 K):

1. Eg(AlAs, Γ) = 3.099 - 8.85×10^{-4} × 300^2 / 830 = 3.003 eV, and Eg(GaAs) = 1.4225 eV.
2. The AlGaAs bowing depends on composition: C = -0.127 + 1.310x = 0.266 eV.
3. Eg = 0.3 × 3.003 + 0.7 × 1.4225 - 0.3 × 0.7 × 0.266 = **1.8408 eV**.
4. The VBO is linear: 0.3 × (-1.33) + 0.7 × (-0.80) = **-0.959 eV** = Ev.
5. Ec = -0.959 + 1.8408 = **0.8818 eV**.
6. Offsets to GaAs: **ΔEc = 0.8818 - 0.6225 = 259 meV**, **ΔEv = -0.800 - (-0.959) = 159 meV**.

Why bowing matters: without it, the band gaps of many alloys would be wrong by tens to hundreds of meV. For indium alloys the VBO bowing is also essential: with it, the lattice-matched pair $In_{0.53}Ga_{0.47}As/In_{0.52}Al_{0.48}As$ gets the measured conduction-band offset of about 0.52 eV.

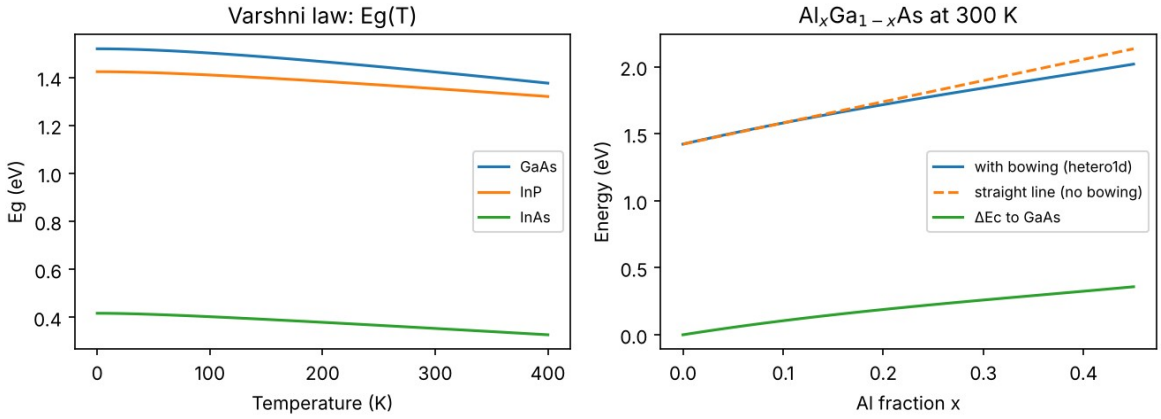


Figure 9.2: left, the band gap versus temperature (Varshni law) for GaAs, InP and InAs. Right, Al_xGa_(1-x)As at 300 K: band gap with bowing (solid) compared with a straight line between GaAs and AlAs (dashed), and the conduction-band offset to GaAs.

9.5 Step 3: quaternaries

Quaternary parameters are interpolated from the ternaries that form their edges in composition space, for example for Ga_x In_(1-x) As_y P_(1-y):

$$Q(x,y) = \frac{x(1-x)\,[\,y\,T_{ABC}(x) + (1-y)\,T_{ABD}(x)\,] + y(1-y)\,[\,x\,T_{ACD}(y) + (1-x)\,T_{BCD}(y)\,]}{x(1-x) + y(1-y)}$$

Why this formula: it is the standard interpolation of Vurgaftman et al., and it reduces exactly to the correct ternary on every edge and to the binaries at the corners. hetero1d checks this automatically: AlGaInAs with x = 0.3, y = 0.7 gives exactly the Eg of $Al_{0.3}Ga_{0.7}As$ (1.8408 eV).

9.6 Step 4: inside the Solver

On the grid, each point gets Ec and Ev from its layer, and electrostatic potentials are added to both (Section 6.1). The quantum states are then computed in this profile:

- electron energies lie above Ec (in the well),
- hole energies lie below Ev,
- all on the same absolute scale as the materials.

9.7 Reading energies correctly

Because everything is on one scale, energies of different kinds can be combined by simple subtraction. The lowest interband transition, for example, is the electron energy minus the hole energy, and it automatically includes the band gap and both confinement energies:

$$E_{e0-hh0} = E_e^{(0)} - E_{hh}^{(0)} = E_g + E_{conf}^{e} + E_{conf}^{hh}$$

```
s = Structure.single_well(well="GaAs", barrier=("AlGaAs", 0.3), Lw=10, Lb=25)
sol = Solver(s, T=300).solve()
sol.Ee[0]                            # absolute: 0.6533 eV for a 10 nm GaAs well
(sol.Ee[0] - sol.Ec.min()) * 1000    # confinement energy: 30.85 meV above the well
bottom
(sol.Ev.max() - sol.Ehh[0]) * 1000   # heavy-hole confinement: 7.8 meV below the
valence band
sol.Ee[0] - sol.Ehh[0]               # e0-hh0 transition: 1.4611 eV (848.5 nm)
```

Note. Only energy **differences** are physical: confinement energies, offsets, transition energies, splittings. Absolute values depend on the chosen reference (InSb valence band = 0) and have no meaning on their own. This is why an electron level of 0.65 eV in GaAs is not "too high": it is 30.85 meV above a conduction band that sits at 0.6225 eV on this scale.

9.8 The same scale for your own materials

When you register a material with `add_material`, place its Ec and Ev on this scale. The easiest way is relative to a built-in material: for a barrier 300 meV above GaAs in the conduction band and 150 meV below it in the valence band, use Ec = 0.6225 + 0.300 = 0.9225 eV and Ev = -0.800 - 0.150 = -0.950 eV (at 300 K).

10. Limits of the model

Knowing why the model works also means knowing where it stops. hetero1d is a single-band effective-mass solver in one dimension:

Not included	Consequence
Strain	strained layers (InGaAs on GaAs, strained InGaAsP) are flagged by describe(); their band edges and masses are approximate
Non-parabolicity	levels high in narrow-gap wells (InAs, InGaAs, InSb) come out somewhat too high
X and L valleys	indirect materials (AlGaAs above x ≈ 0.45, AlAs, AlSb, GaP, high-Al AlGaInP) lack their lowest conduction states
Valence-band mixing	in-plane hole dispersion and optical gain are approximate; exciton binding energies are a lower bound
Polarisation in wurtzite nitrides	built-in fields must be added with V_ext
Temperature dependence of custom materials	add_material values are used at every temperature

Within these limits the solver is accurate to better than 0.04 meV against exact solutions, reproduces textbook results, and gives reliable trends for design.

Appendix: quick reference

```
# materials
material(name, x=None, y=None, T=300.0)            -> dict
list_materials()
add_material(name, Ec, Ev, me, mhh, mlh, eps_s, eps_inf=None, hwLO=0.036, a=5.65325,
g=2.0, mhh_par=None, mlh_par=None)
lattice_matched(name, substrate="InP", fixed=None, T=300.0)

# structure
Structure(layers=None, substrate="GaAs", name="structure")
  .add(material, thickness, x=None, doping=0.0, y=None)
  .add_graded(material, thickness, x=None, y=None, slice_nm=0.2, doping=0.0,
label=None)
  .describe(T=300.0)
  .single_well / multi_well / double_well / superlattice / step_well
  .parabolic_well / triangular_well / v_well / graded_well / multi_graded_well /
sawtooth / from_profile

# solver
Solver(structure, T=300.0, field=0.0, dz=0.1, Bz=0.0, Bpar=0.0, z0=None, V_ext=None)
  .solve(n_e=6, n_h=6)
  .schrodinger_poisson(n_e=8, mix=0.1, tol=1e-5, max_iter=600, verbose=True,
right_bc="neumann")
  results: z, Ec, Ev, Ee, Ehh, Elh, psi_e, psi_hh, psi_lh, e_bound, hh_bound, lh_bound
  analysis: intersubband, interband, isb_absorption, interband_absorption, exciton,
fermi_level,
            lo_phonon_rates, transmission, minibands, layer_probabilities,
expect_z, ...
  fields: stark_sweep, landau_levels, landau_fan, fermi_level_B, zeeman, g_factor,
inplane_dispersion
  output: report, plot_summary, plot_bands, plot_stark, plot_magnetic

# energy scale
Ev = VBO;  Ec = VBO + Eg(T);  Eg(T) = Eg(0) - alpha T^2/(T + beta);  alloys: x PA +
(1-x) PB - x(1-x) C
```